

ML-REFLECTOMETRY WORKSHOP

Introduction & ML supported reflectometry analysis

October 21st 2026 | Stefan Häusler, Mykhailo Levytskyi | JCNS

Organization

09:30 – 12:30 Conventional reflectometry analysis

12:30 – 14:00 Lunch

14:00 – 15:30 Foundations of machine learning and neural networks

15:30 – 17:00 Advanced analysis methods for reflectivity data

17:00 – 17:30 Q&A

Outline

Foundations of machine learning and neural networks

Advanced analysis methods for reflectivity data

Machine learning in reflectometry

Physics-based fitting and MCMC already support accurate parameter inference.

Conventional inference

Each new measurement requires inference using the physical forward model.

Exploring uncertainty can require many model evaluations.

Learning from simulations

Generate examples and train a network upfront.

Reuse the trained network for fast approximate predictions on new curves.

Amortization: share the initial cost across many measurements.

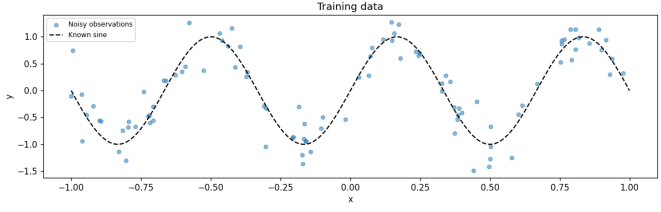
Simulated training data do not enforce physics in every prediction. Physical checks and refinement remain important.

Supervised learning

Examples pair an input with an observed or simulated target:

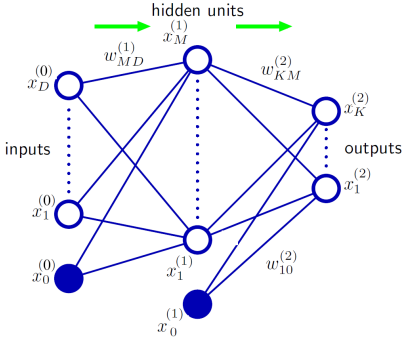
$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad \hat{y} = f_{\mathbf{w}}(\mathbf{x}).$$

	Input	**Target**
Exercise 1	Coordinate x	Noisy observed y
Exercise 2	Reflectivity curve	Layer SLD ρ



Training adjusts **w** using examples. Prediction applies the learned mapping to a new input.

A neuron and a neural network



Inputs, hidden units, outputs

One hidden unit in Exercise 1:

$$h_j(x) = g(w_j x + b_j).$$

Nonlinear activation:

$$g(z) = \text{sigmoid}(z) = \frac{1}{1 + e^{-z}}.$$

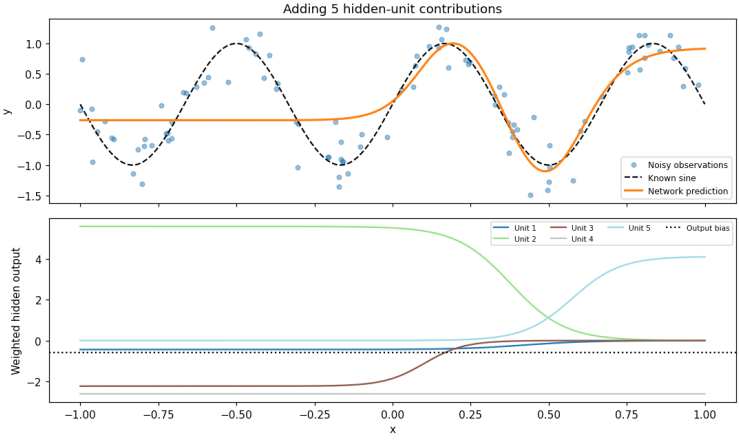
Linear output:

$$\hat{y} = b_{\text{out}} + \sum_{j=1}^{M} v_j h_j(x).$$

w collects all trainable weights and biases.

Without a nonlinear activation, stacked affine layers still give an affine function.

Function approximation



Add the contributions

Weighted sigmoids plus the output bias form the prediction.

More units provide more contributions to arrange.

Expressive power gives no recipe for finding useful weights.

Representing a solution, finding it, and generalizing are separate questions.

Learning objective and likelihood

Squared discrepancies to observed targets:

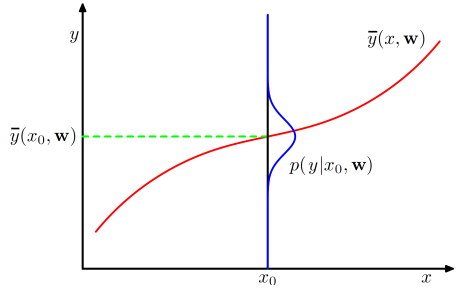
$$\mathcal{L}_{\text{train}}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\big(f_{\mathbf{w}}(\mathbf{x}_i) - y_i\big)^2.$$

A probabilistic interpretation:

$$q_{\mathbf{w}}(y \mid \mathbf{x}) = \mathcal{N}\Big(y; f_{\mathbf{w}}(\mathbf{x}), \sigma_0^2\Big).$$

For fixed Gaussian variance,

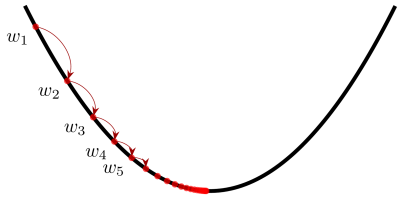
$$-\frac{1}{N}\sum_i \log q_{\mathbf{w}}(y_i \mid \mathbf{x}_i) = \frac{\mathcal{L}_{\text{train}}}{2\sigma_0^2} + C.$$



A conditional distribution at one input

- Minimizing MSE maximizes the likelihood (ML) for this fixed-variance Gaussian model.
- Maximizing the expected log-likelihood minimizes the KL divergence between the true and model distributions.
- The MSE-optimal prediction is the conditional mean $\mathbb{E}[y \mid \mathbf{x}]$, also for non-Gaussian targets (for finite variance).

Gradient descent and backpropagation



Schematic loss as a function of a weight

Gradient-descent principle

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \, \nabla_{\mathbf{w}} \mathcal{L}_{\text{train}}(\mathbf{w}_t).$$

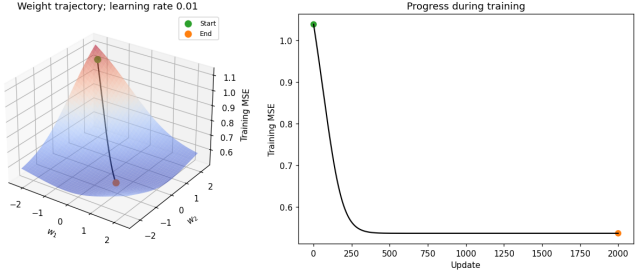
1. Predict and calculate the loss.
2. Calculate gradients efficiently (Backpropagation).
3. Update weights using an optimizer.

The learning rate η controls the adjustment scale. Too small can be slow, too large can overshoot or stall.

The notebooks use **Adam**, which adapts gradient-based updates. One epoch processes all training examples.

Exercise 1: how a network learns

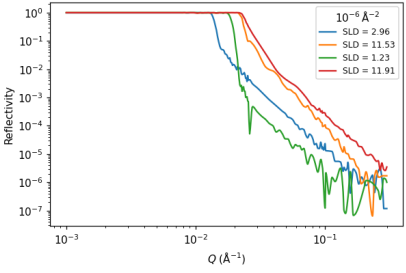
1. Compare hidden-unit contributions for 3, 5, and 10 units.
2. Change initialization while keeping the observations fixed.
3. Watch two weights move and compare learning rates.



Two adjustable weights make the learning process visible.

Open `Ex1_neural_networks.ipynb` **and work through both parts.**

Statistical inverse learning



Simulated curves with known layer SLD

Physical forward model

$$\boldsymbol{\theta} \longmapsto R(Q)$$

SLD marginal posterior

$$q_{\mathbf{w}}(\rho \mid \mathbf{R}) \approx p_{\mathrm{sim}}(\rho \mid \mathbf{R}).$$

Thickness and roughness also vary. The curve is the only input.

A point prediction summarizes possible targets:

$$\hat{\rho}(\mathbf{R}) \text{ targets } \mathbb{E}_{\mathrm{sim}}[\rho \mid \mathbf{R}].$$

Simulation sampling supplies implicit prior information. An average of alternatives need not identify a compatible structure.

Training, validation, and testing

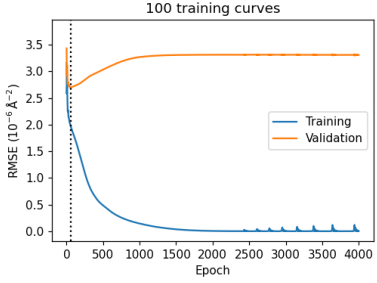
Training 1,901 curves	Fit network weights by minimizing $\mathcal{L}_{\text{train}}$.
Validation 634 curves	Compare settings and select a checkpoint using $\mathcal{L}_{\text{val}}$.
Test 634 curves	Evaluate the selected model using $\mathcal{L}_{\text{test}}$.

Test loss estimates expected loss on new examples from the same population.

Selection uses validation data, including the stopping epoch.

Ex2 uses a 60/20/20 split. Fit scaling on training data and apply it to all partitions.

Overfitting



Training error keeps falling while validation error rises.

Select the checkpoint at the validation minimum. The final epoch can give worse predictions on new curves.

Dotted lines mark selected checkpoints.

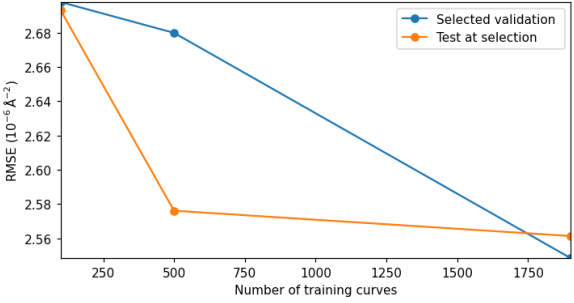
Exercise 2: predicting layer SLD

1. Train a point predictor and compare it with the training-mean baseline.
2. Inspect generalization as training duration and sample count change.
3. Continue until Gaussian prediction of mean and variance.

Common checkpoint: explain why the selected model can differ from the final epoch.

Open `Ex2_reflectometry.ipynb`**.**

Overfitting and training-set size



At the selected checkpoint

Compare validation and test error as training size changes.

More representative data can help generalization.

More data helped in this saved run. Improvement need not be monotonic. The notebook also supplies a training-mean baseline.

A Gaussian prediction: mean and variance

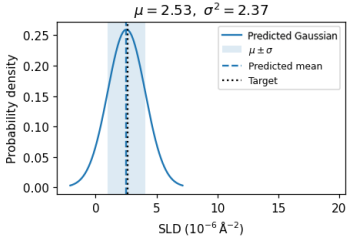
Learn an input-dependent centre and spread

$$q_{\mathbf{w}}(\rho \mid \mathbf{R}) = \mathcal{N}\Big(\rho; \mu_{\mathbf{w}}(\mathbf{R}), \sigma^2_{\mathbf{w}}(\mathbf{R})\Big).$$

For example, Gaussian negative log-likelihood is

$$\ell_{\mathrm{NLL}} = \frac{1}{2}\left[\log \sigma^2 + \frac{(\rho - \mu)^2}{\sigma^2}\right] + C.$$

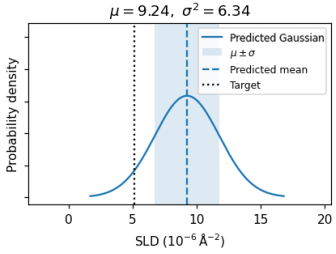
Confident misses incur a large residual penalty. Very wide predictions incur a logarithmic penalty.



The predicted spread is intended to estimate **aleatoric uncertainty**. It may also absorb model error.

The targets are conditional mean and variance. One Gaussian cannot represent separated solutions.

Predicted uncertainty and coverage



The target is about 1.64σ from the mean.

Coverage across 634 test curves

Interval	Gaussian	Observed
$\mu \pm \sigma$	68.3%	69.1%
$\mu \pm 1.96\sigma$	95.0%	95.6%

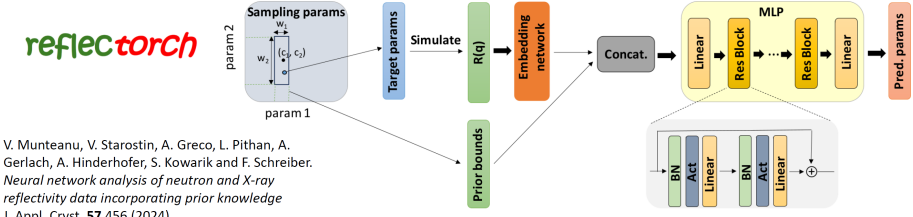
Compare coverage **and interval width**. Wide intervals can cover many targets while giving little information.

A standard deviation describes spread, not an error bound.

Global coverage checks collective frequencies within the simulation population. It does not validate each individual uncertainty estimate.

NN for neutron reflectivity

Previous implementation



V. Munteanu, V. Starostin, A. Greco, L. Pithan, A. Gerlach, A. Hinderhofer, S. Kowarik and F. Schreiber. *Neural network analysis of neutron and X-ray reflectivity data incorporating prior knowledge* J. Appl. Cryst. **57** 456 (2024)

- **Deep neural networks**: rapid generation of single candidate solutions.
- **Prior amortization:** the same trained network supports different prior choices
- **Anaklasis**: downstream parameter optimization via conventional iterative fitting

Learning the joint parameter posterior

SLD, thickness, and roughness can be correlated.

One SLD value	Conditional mean
Gaussian SLD density	Conditional mean and variance
Joint density	Correlations and separated solution families

PANPE: Prior-Amortized Neural Posterior Estimation

A neural network approximates the joint posterior of the physical parameters, using the measurement **R** and a chosen prior φ.

- θ: physical parameters to infer, such as SLD, thickness, and interface roughness.
- φ: parameters defining the chosen prior, such as lower and upper bounds on θ.

$$q_{\mathbf{w}}(\theta \mid \mathbf{R}, \varphi) \approx p(\theta \mid \mathbf{R}, \varphi).$$

Prior amortization: the same trained network supports different prior choices.

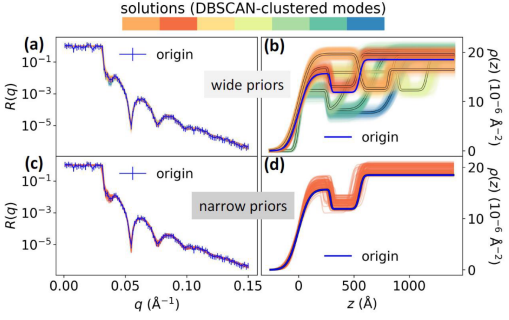
Outline

Foundations of machine learning and neural networks

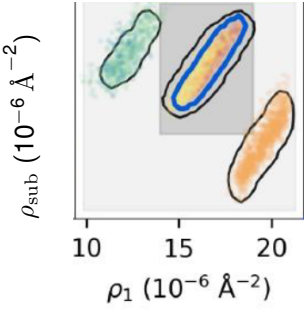
Advanced analysis methods for reflectivity data

JÜLICH Forschungszentrum

Correlations and alternative solutions



Similar curves, different depth profiles.



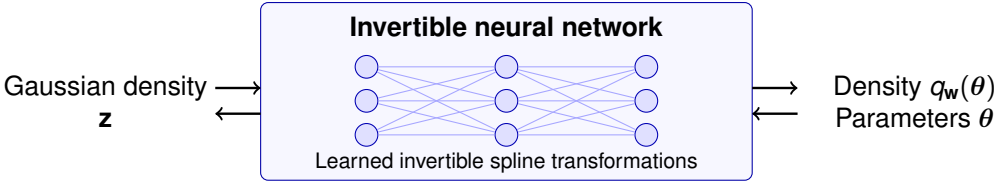
Layer SLD and substrate SLD vary together.

We need their joint distribution, including correlations.

Published PANPE-IS example: Starostin et al., Science Advances (2025).

The flow is an invertible neural network

Familiar: learned weights, nonlinear activations, backpropagation.



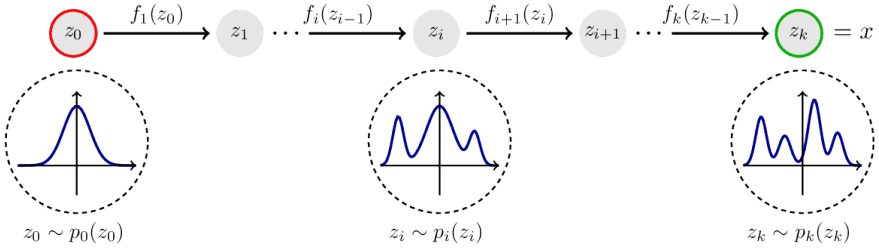
Sampling: $\mathbf{z} \to \theta$. Training evaluates $\theta \to \mathbf{z}$.

New: reversible layers with learned nonlinear spline functions.

Train: give high probability density to known simulated structures.

$$\ell = -\log q_{\mathbf{w}}(\theta) = \underbrace{\tfrac{1}{2}\|\mathbf{z}\|^2}_{\text{Gaussian score}} - \underbrace{\log\left|\det \frac{\partial \mathbf{z}}{\partial \theta}\right|}_{\text{stretch/compression correction}} + C, \qquad \mathbf{z} = f_{\mathbf{w}}^{-1}(\theta)$$

From a Gaussian to a joint distribution



One Gaussian draw becomes one complete parameter vector.

Stretching lowers density; compression raises it. Probability is conserved.

Gaussian dimension = sampled parameters: **7** (one layer), **10** (two layers).

The curve and priors condition the flow

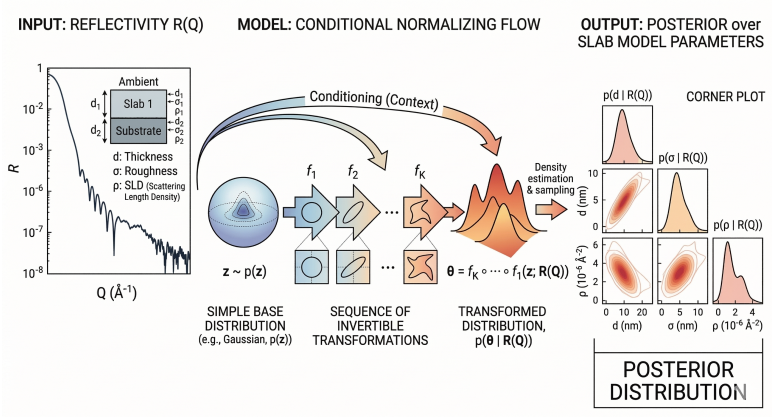
Keep fixed

The measured curve and chosen prior bounds.

Draw again

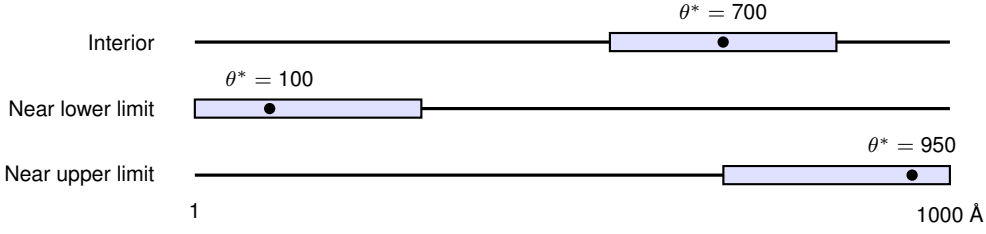
A new Gaussian vector gives another complete structure.

The trained weights stay the same.



Changing priors after training

The same trained network receives different prior bounds as context. For thickness in this exercise: $L = 1$ and $U = 1000$ Å.



The window width is $0.30(U - L) = 299.7$ Å; it shifts at a boundary.

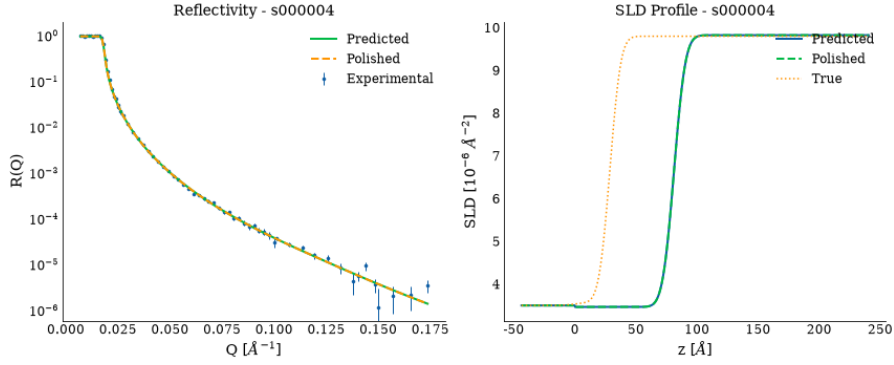
Wider bounds change the supplied information; they add no measurements.

Notebook 01: one experiment, joint parameters

1. Load the curve, errors and supplied reference structure.
2. Inspect preprocessing and the actual prior bounds.
3. Load the pretrained one-layer model and draw parameter vectors.
4. Compare the selected curve, SLD profile and individual parameters.

Discuss: Which information comes from the measurement, and which have we supplied?

Notebook 01: one experiment, joint parameters



Existing instructor output: `s000004`, 30% prior setting.

Does matching a curve establish a unique structure?

Notebook 01: explore prior and experiment choice

1. Keep the experiment and draw count fixed. Compare 10%, 30% and 60% prior settings using the displayed bounds, parameters and profiles.
2. Choose another one-layer experiment and keep the settings fixed. Which features of the result remain similar, and which change?

Optional: plot sampled thickness against layer SLD.

Do those parameters vary together? Separate means and SDs do not show their relationship.

Notebook 02: measurement and material information

Si / SiO_2 / polymer / D_2O

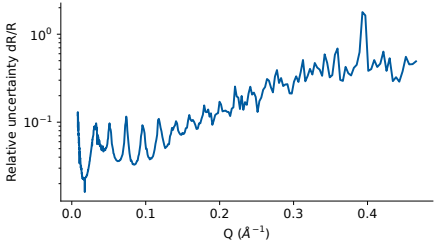
Load Q, R, dR, dQ and set two-layer priors.

SiO_2 SLD: $[3.46, 3.48]$

Polymer SLD: $[-0.5, 3.0]$

D_2O SLD: $[6.34, 6.36]$

SLD units: 10^{-6} Å^{-2}.



dR: reflectivity uncertainty.
dQ: Q resolution and curve smearing.

Discuss: Would a narrow inferred SLD spread necessarily come from the data?

Notebook 02: what changes with more draws?

For the same ORSO curve and prior bounds, compare 500 and 2,000 NF draws.

- Reset the seed as specified in the existing activity.
- Compare the selected parameters, sample summaries and SLD overlays.
- Do visible families change, or become more densely sampled?

Discuss: Must the inferred structural spread shrink with more draws?

This checks finite-sample stability. More draws do not add experimental information.

Notebook 03: same sample, separate inferences

Optional example: two neutron polarisation channels of air / Pt / Co / Al_2O_3.

Shared geometry	Layer thicknesses and interface roughnesses
Effective SLD	Co SLD includes a channel-dependent magnetic contribution
Independent runs	Each channel is inferred separately; shared geometry is not enforced

Discuss: Is comparison with a joint XRR + PNR fit a direct test of which method is more accurate?

Thank you for your attention!